EXPERIMENT – 5

Title

SQL Views, View Updatability and Recursive CTE

Aim

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display employee reporting chains.

Objectives

1. To understand the concept of SQL views.
2. To create a view for department salary summary.
3. To create a view for employee hierarchy.
4. To test the updatability of SQL views.
5. To understand recursive Common Table Expressions.
6. To display employee reporting chains using a recursive CTE.

Software / Tools Required

- ByteXL / Nimbus
- MySQL
- SQL

Theory

A **view** is a virtual table created from the result of an SQL query. It does not normally store the data separately. Instead, it displays data from one or more underlying tables.

Naman Verma
25LBCS3276

Views are useful for simplifying complex queries, improving data security, and presenting only required information to users.

1. Department Salary Summary View

A department salary summary view can be created using aggregate functions such as `COUNT()`, `SUM()`, and `AVG()`. It can display the number of employees, total salary, and average salary for each department.

2. Employee Hierarchy View

An employee hierarchy represents the relationship between employees and their managers. A self-referencing `ManagerID` column can be used to identify the manager of each employee.

3. View Updatability

Some SQL views are updatable, while others are not.

A simple view based on one table and without aggregate functions or grouping can generally be updated. Views containing `GROUP BY`, aggregate functions, joins, or other complex operations are generally not directly updatable.

4. Recursive CTE

A **Recursive Common Table Expression (CTE)** is used to process hierarchical or tree-like data. It consists of two parts:

- Anchor query – provides the starting records.
- Recursive query – repeatedly finds related records.

A recursive CTE can be used to display an employee's reporting chain from the top-level manager to lower-level employees.

Naman Verma
25LBCS3276

SQL Implementation

1. Add ManagerID to Employee Table

The `ManagerID` column is required to represent the employee-manager relationship.

```
ALTER TABLE Employee
ADD ManagerID INT NULL;
```

2. Add Self-Referencing Foreign Key

```
ALTER TABLE Employee
ADD CONSTRAINT fk_manager
FOREIGN KEY (ManagerID) REFERENCES Employee(EmpID);
```



3. Assign Managers

```
UPDATE Employee
SET ManagerID = CASE
    WHEN EmpID IN (1, 6, 11, 16, 21) THEN NULL
    WHEN EmpID BETWEEN 2 AND 5 THEN 1
    WHEN EmpID BETWEEN 7 AND 10 THEN 6
    WHEN EmpID BETWEEN 12 AND 15 THEN 11
    WHEN EmpID BETWEEN 17 AND 20 THEN 16
    WHEN EmpID BETWEEN 22 AND 25 THEN 21
    WHEN EmpID = 26 THEN 1
    WHEN EmpID = 27 THEN 6
    WHEN EmpID = 28 THEN 11
```

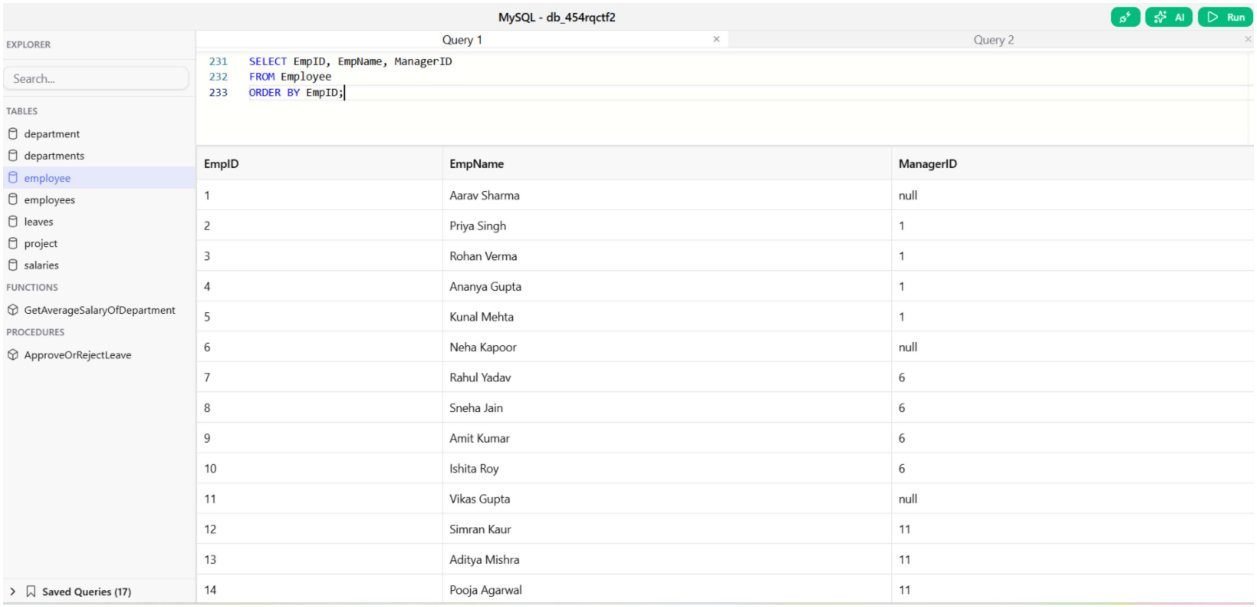
Naman Verma
25LBCS3276

```
    WHEN EmpID = 29 THEN 16
    WHEN EmpID = 30 THEN 21
END;
```

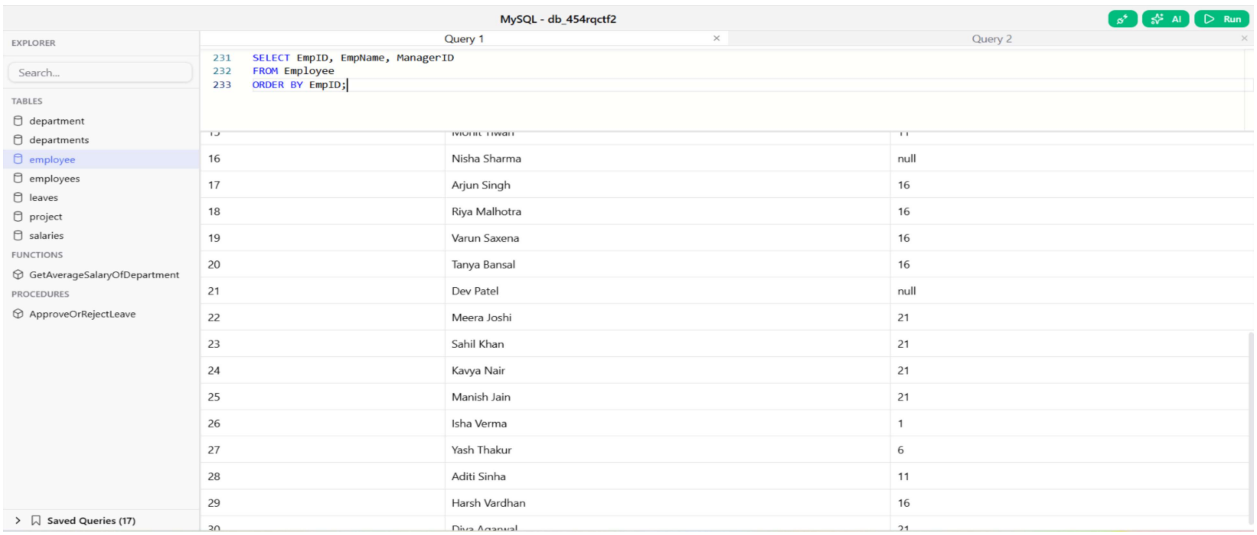


4. Display Employee Manager Relationship

```
SELECT EmpID, EmpName, ManagerID
FROM Employee
ORDER BY EmpID;
```



Naman Verma
25LBCS3276



5. Create Department Salary Summary View

This view displays the number of employees, total salary, and average salary for each department.

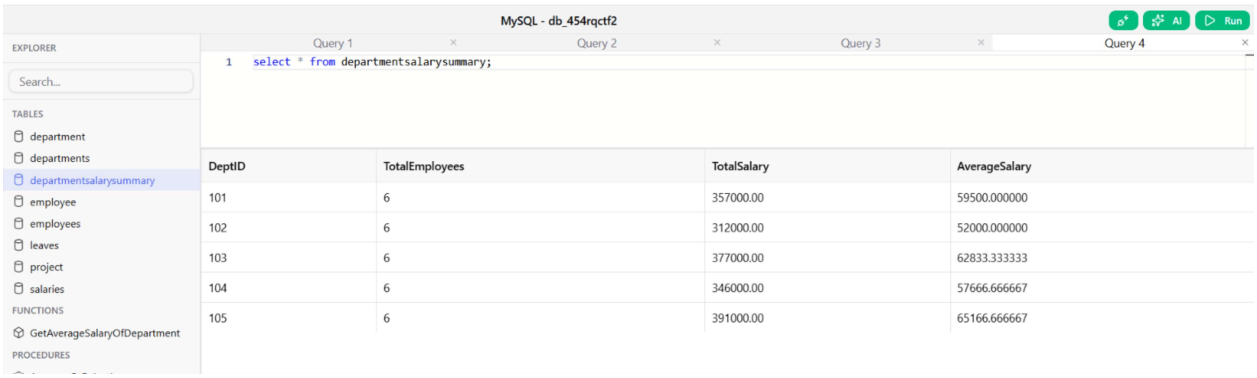
```
CREATE VIEW DepartmentSalarySummary AS
SELECT DeptID,
       COUNT(*) AS TotalEmployees,
       SUM(Salary) AS TotalSalary,
       AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY DeptID;
```



6. Display Department Salary Summary

```
SELECT * FROM DepartmentSalarySummary;
```

Naman Verma
25LBCS3276



7. Create Employee Hierarchy View

This view displays each employee along with the name of their immediate manager.

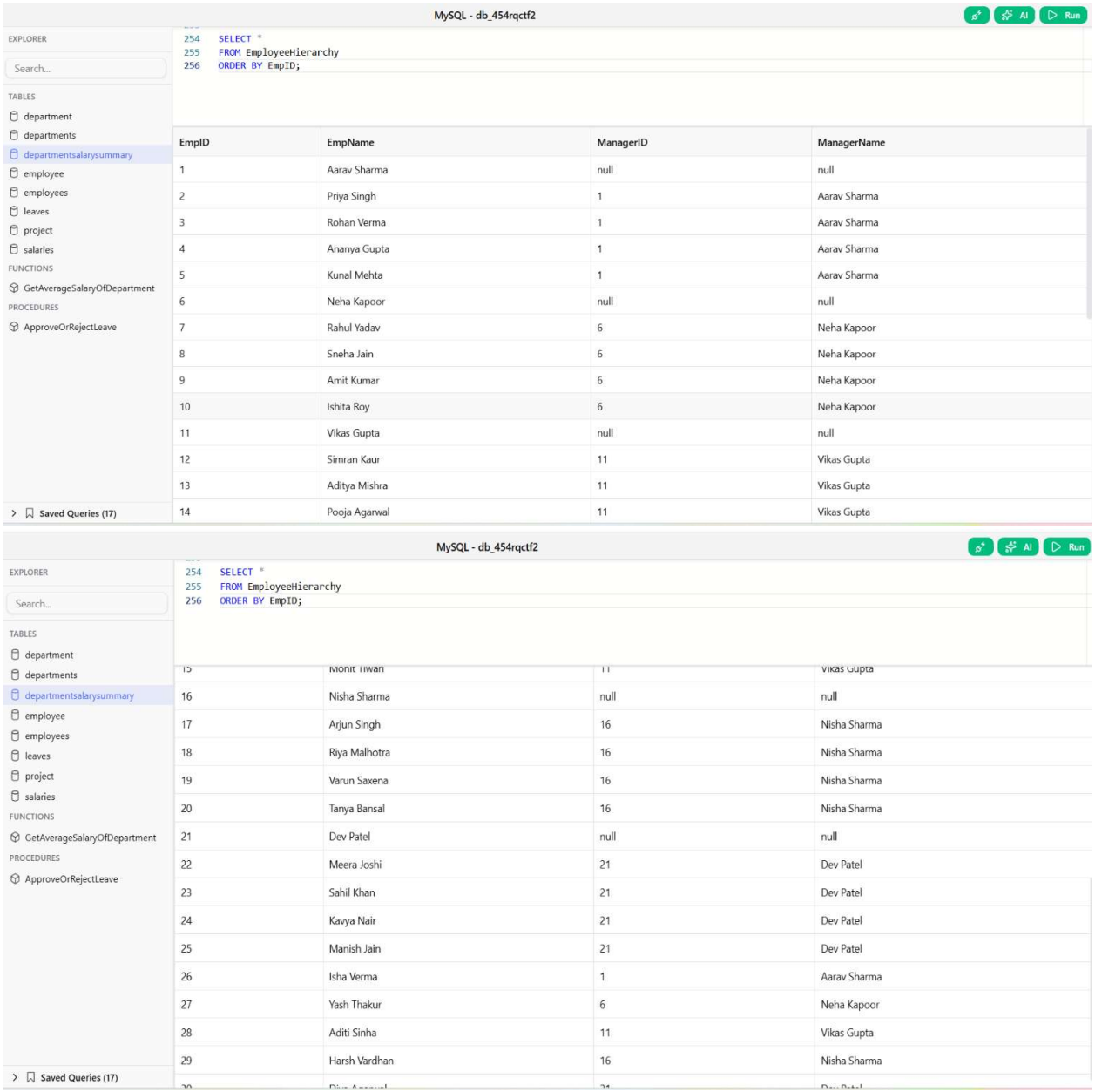
```
CREATE VIEW EmployeeHierarchy AS
SELECT E.EmpID,
       E.EmpName,
       E.ManagerID,
       M.EmpName AS ManagerName
FROM Employee E
LEFT JOIN Employee M
ON E.ManagerID = M.EmpID;
```



8. Display Employee Hierarchy

```
SELECT *
FROM EmployeeHierarchy
ORDER BY EmpID;
```

Naman Verma
25LBCS3276

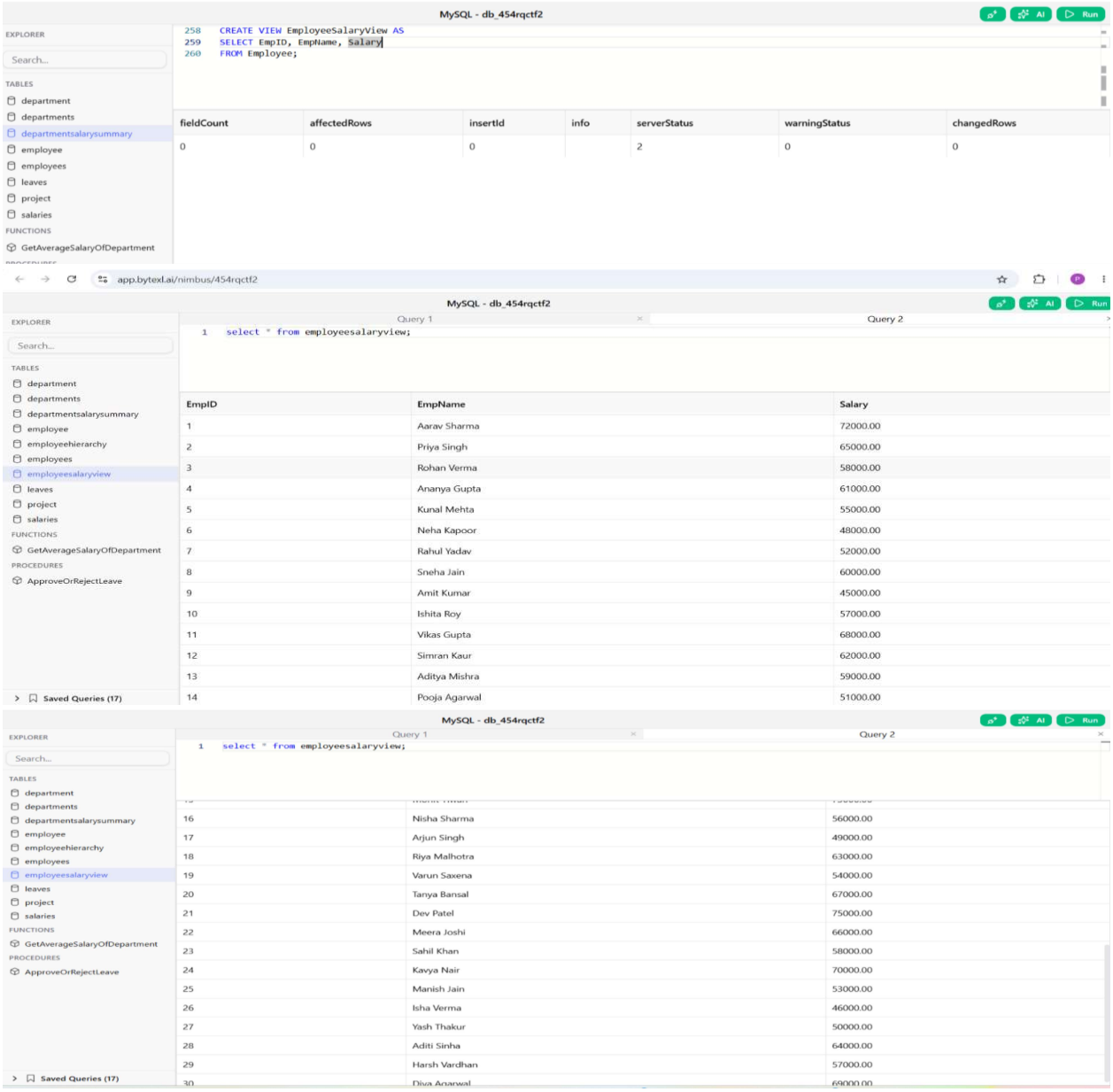


9. Test Updatability of Simple View

First, create a simple view containing employee ID, name, and salary.

```
CREATE VIEW EmployeeSalaryView AS
SELECT EmpID, EmpName, Salary FROM Employee;
```

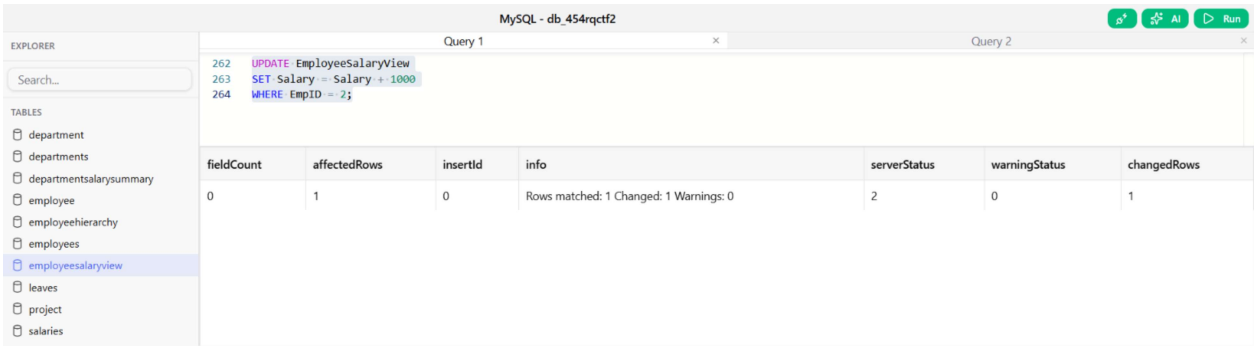
Naman Verma
25LBCS3276



10. Update the Updatable View

```
UPDATE EmployeeSalaryView
SET Salary = Salary + 1000
WHERE EmpID = 2;
```

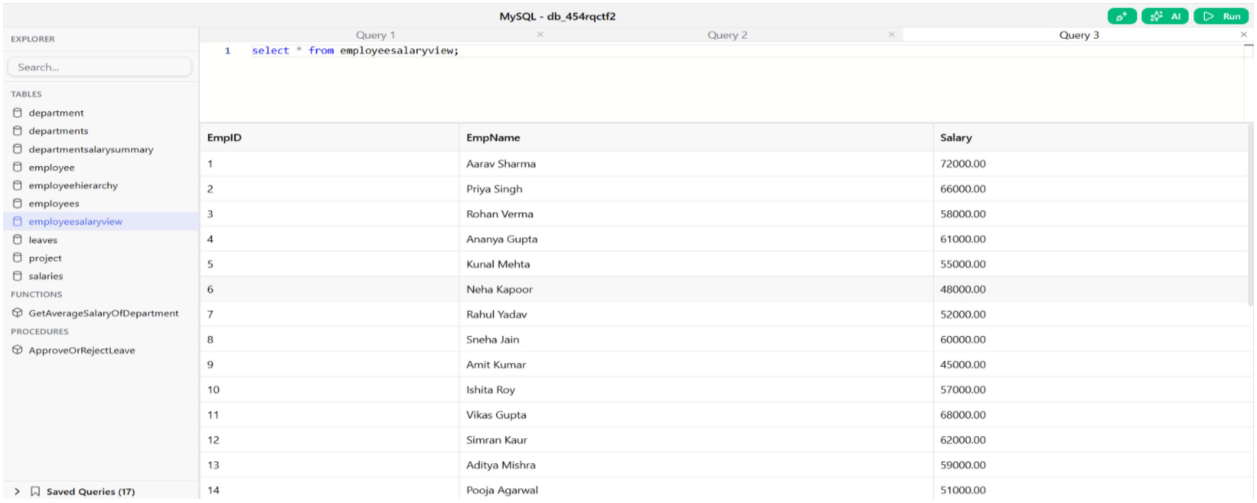
Naman Verma
25LBCS3276



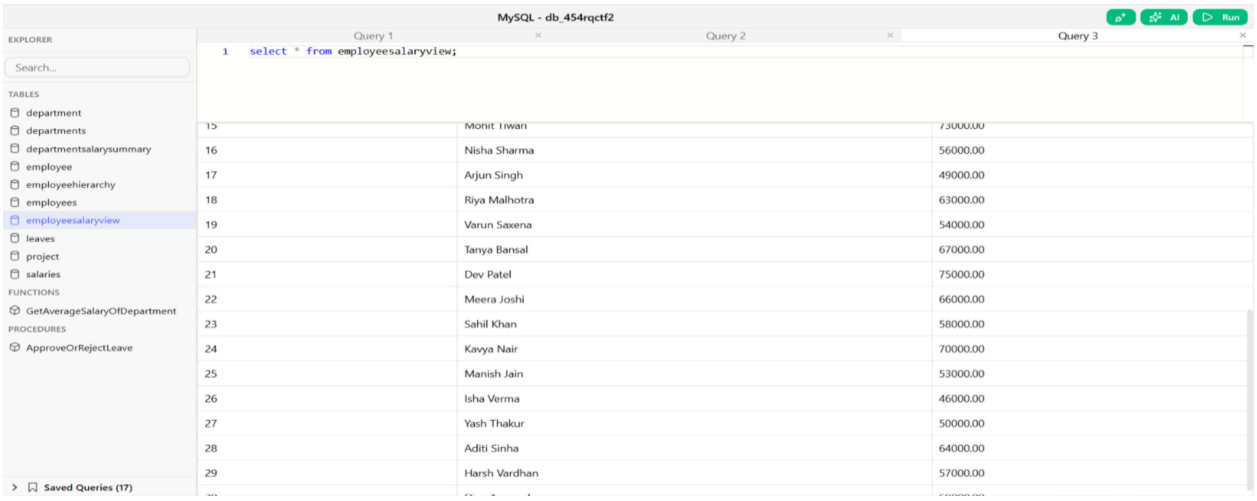
11. Verify Updated Data

```
SELECT *
FROM EmployeeSalaryView
WHERE EmpID = 2;
```

The simple view can be updated because it is based on a single table and does not contain grouping or aggregate functions.



Naman Verma
25LBCS3276



12. Test Department Salary Summary View

The following statement demonstrates that an aggregate view is not directly updatable.

```
UPDATE DepartmentSalarySummary
SET AverageSalary = AverageSalary + 1000
WHERE DeptID = 101;
```

This view is not directly updatable because it contains aggregate functions and `GROUP BY`.

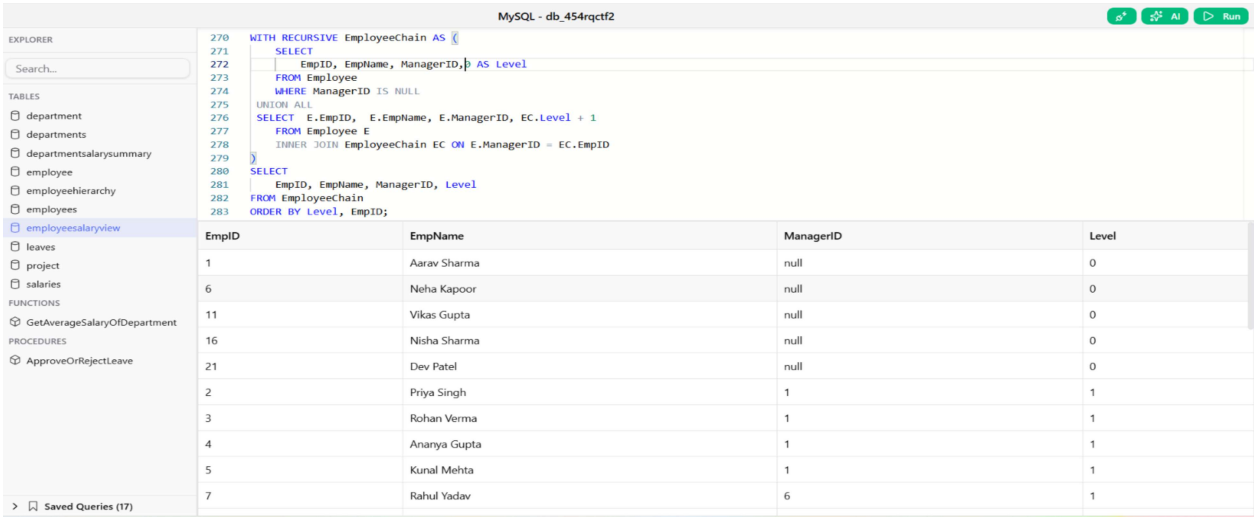


13. Recursive CTE for Reporting Chain

The following recursive CTE displays employees along with their reporting levels.

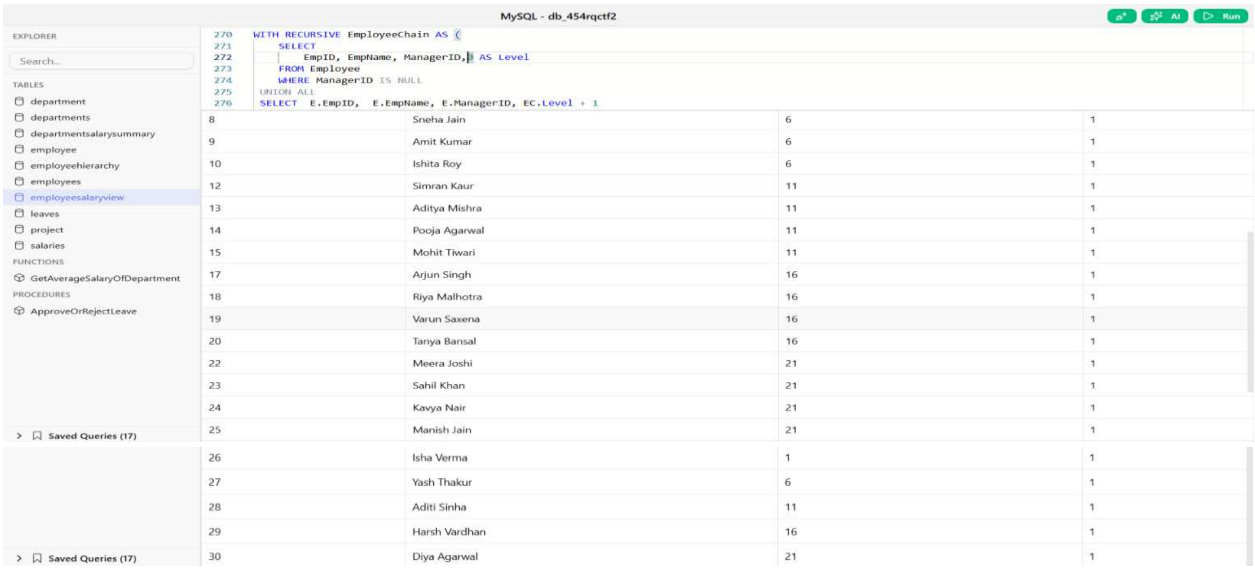
```
WITH RECURSIVE EmployeeChain AS (
    SELECT
        EmpID,
        EmpName,
```

```
        ManagerID,
        0 AS Level
    FROM Employee
    WHERE ManagerID IS NULL
 UNION ALL
SELECT
        E.EmpID,
        E.EmpName,
        E.ManagerID,
        EC.Level + 1
    FROM Employee E
    INNER JOIN EmployeeChain EC
    ON E.ManagerID = EC.EmpID
)
SELECT
    EmpID,
    EmpName,
    ManagerID,
    Level
FROM EmployeeChain
ORDER BY Level, EmpID;
```



EmpID	EmpName	ManagerID	Level
1	Aarav Sharma	null	0
6	Neha Kapoor	null	0
11	Vikas Gupta	null	0
16	Nisha Sharma	null	0
21	Dev Patel	null	0
2	Priya Singh	1	1
3	Rohan Verma	1	1
4	Ananya Gupta	1	1
5	Kunal Mehta	1	1
7	Rahul Yadav	6	1

Naman Verma
25LBCS3276

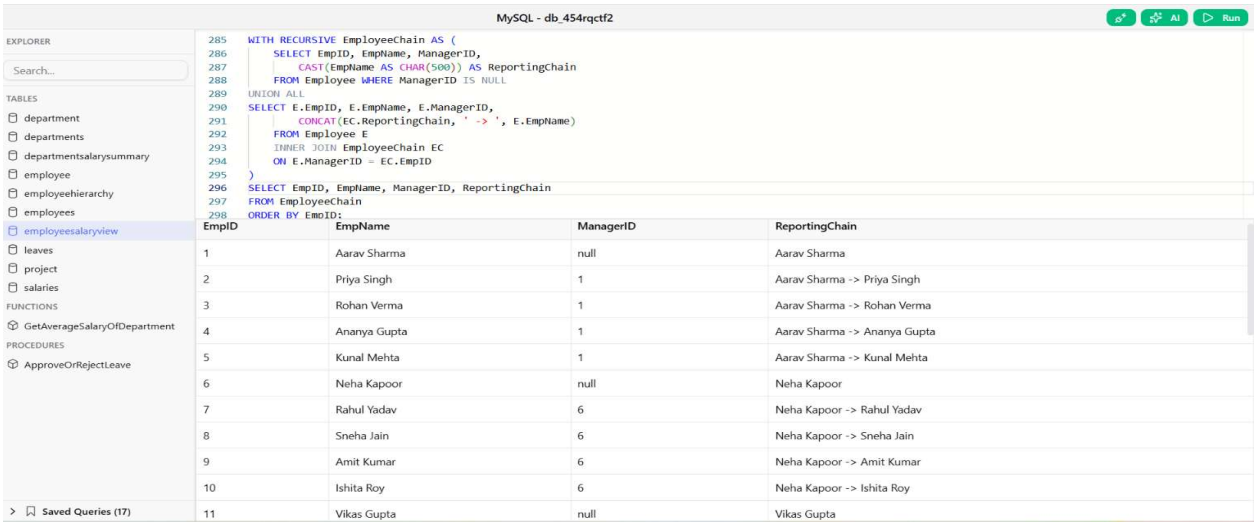


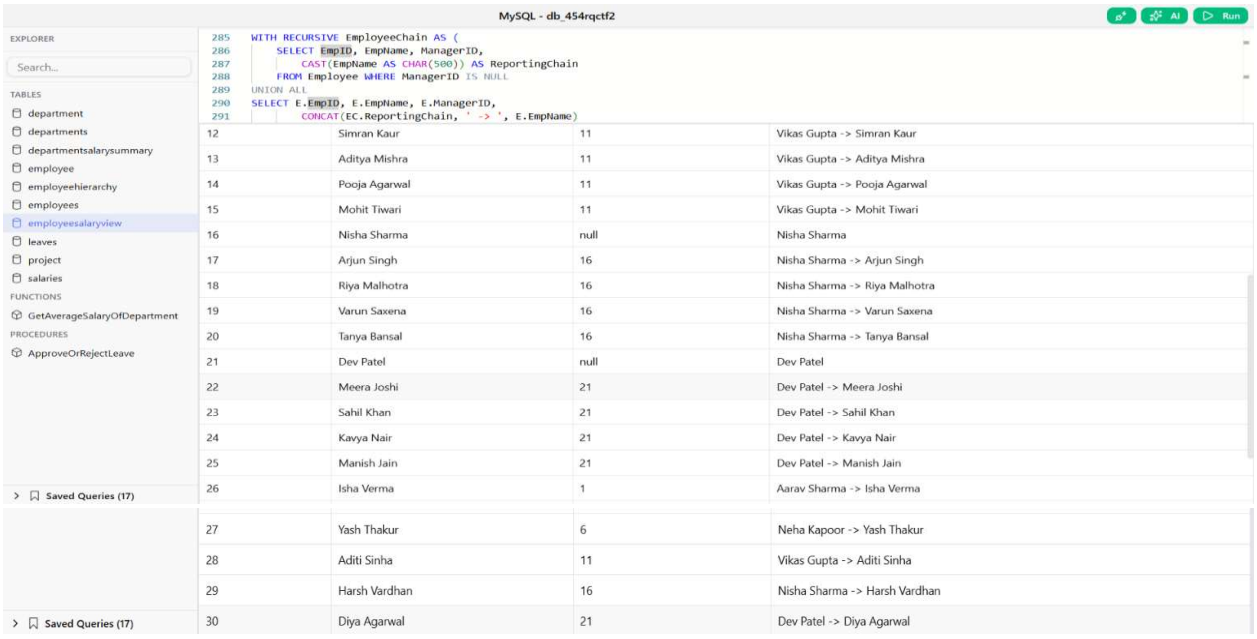
14. Recursive CTE with Reporting Chain

This query displays the reporting hierarchy in a readable form.

```
WITH RECURSIVE EmployeeChain AS (
    SELECT EmpID, EmpName, ManagerID,
        CAST(EmpName AS CHAR(500)) AS ReportingChain
    FROM Employee
    WHERE ManagerID IS NULL
UNION ALL
SELECT
        E.EmpID, E.EmpName, E.ManagerID,
        CONCAT(EC.ReportingChain, ' -> ', E.EmpName)
    FROM Employee E
    INNER JOIN EmployeeChain EC
    ON E.ManagerID = EC.EmpID
)
SELECT
    EmpID,EmpName,ManagerID, ReportingChain
FROM EmployeeChain
ORDER BY EmpID;
```

Naman Verma
25LBCS3276





Result

SQL views for department salary summary and employee hierarchy were successfully created. The updatability of simple and aggregate views was tested. A recursive CTE was successfully implemented to display employee reporting levels and reporting chains.

Learning Outcomes

After completing this experiment, we are able to:

1. Create and use SQL views.
2. Create summary views using aggregate functions.
3. Represent employee-manager relationships.
4. Understand which views are updatable.
5. Use recursive CTEs for hierarchical data.
6. Display employee reporting chains.

Conclusion

This experiment provided practical knowledge of SQL views, view updatability, employee hierarchies, and recursive CTEs. The required views were created successfully, their behavior was tested, and employee reporting chains were displayed using recursive SQL queries.

Naman Verma
25LBCS3276